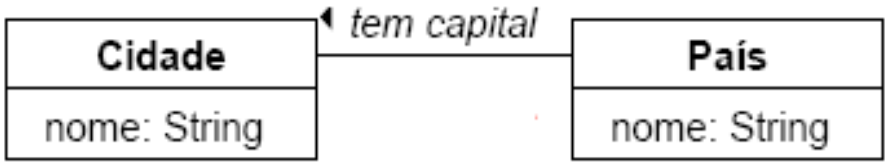
Aula 12 - Relações

Programação Orientada a Objetos
Prof. Matheus Avila Moreira de Paula
matheusavila@cefetmg.br

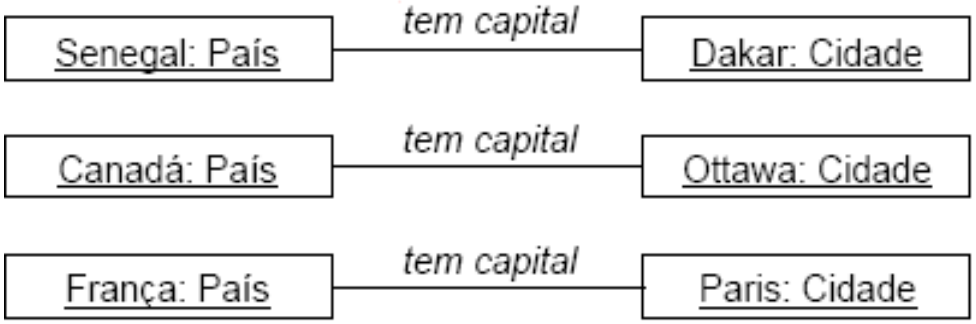
Engenharia de Computação
Centro Federal de Educação Tecnológica Leopoldina

Relacionamento Entre Classes

- Objetos isolados não fazem sentido
- Objetos precisam estar relacionados a outros objetos



Nível de Classes



Nível de Instâncias

UML -Unified Modeling Language

UML – Linguagem de Modelagem Unificada

- Padronização – notação simplificada para:
  - Especificar
  - Visualizar
  - Documentar
  - Construir artefatos

  - Pode ser usada em todos os processos ao longo do ciclo de desenvolvimento

UML – Linguagem de Modelagem Unificada

- Diagrama de Classe
- Diagrama de Caso de Uso
- Diagrama de Sequência
- Diagrama de Colaboração
- Diagrama de Estado
- Diagrama de Atividade
- Diagrama de Componente
- Diagrama de Implantação

Associação

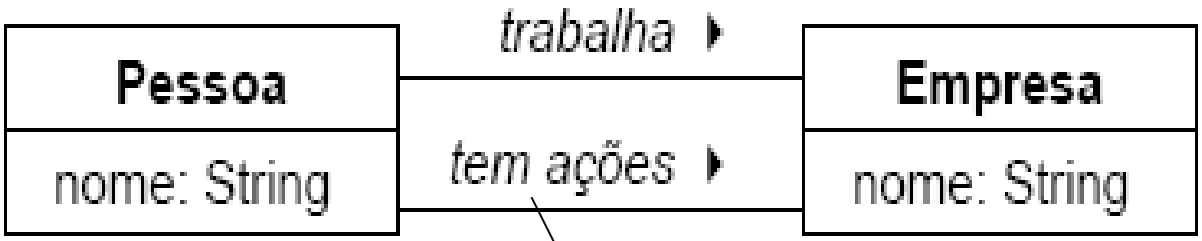
Associação

- Definição
  - É um relacionamento entre objetos.
  - É um relacionamento estrutural
    - Objetos de um elemento estão conectados a objetos de outro elemento

- Detalhamento de uma associação
  - Papéis
  - Multiplicidade
  - Grau
  - Atributo

Nomeação

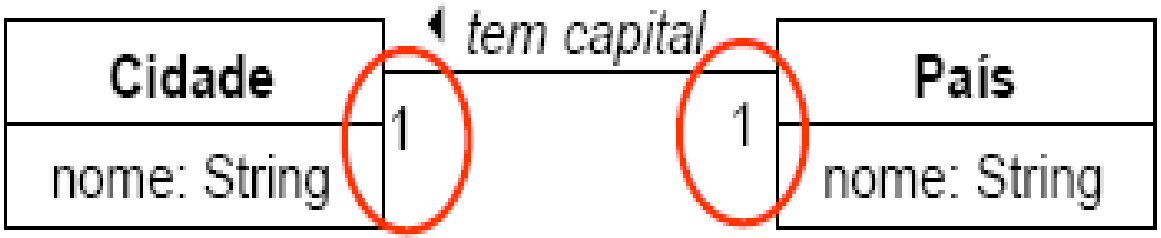
- Podem existir várias associações entre as mesmas classes
  - É preciso nomeá-las de forma diferente
  - Evitar ambiguidade

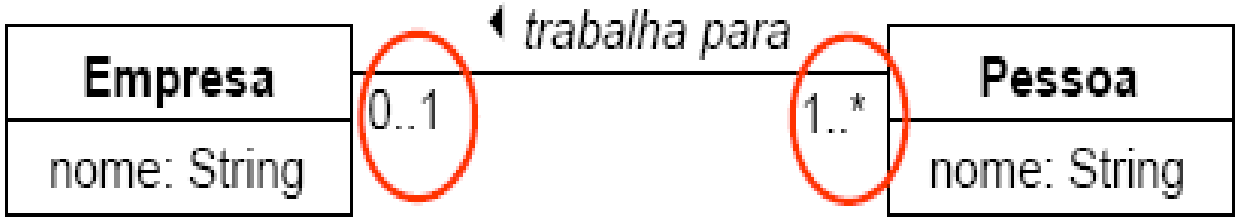


Nome da associação: usado para definir, ou qualificar, a natureza da relação entre as classes associadas

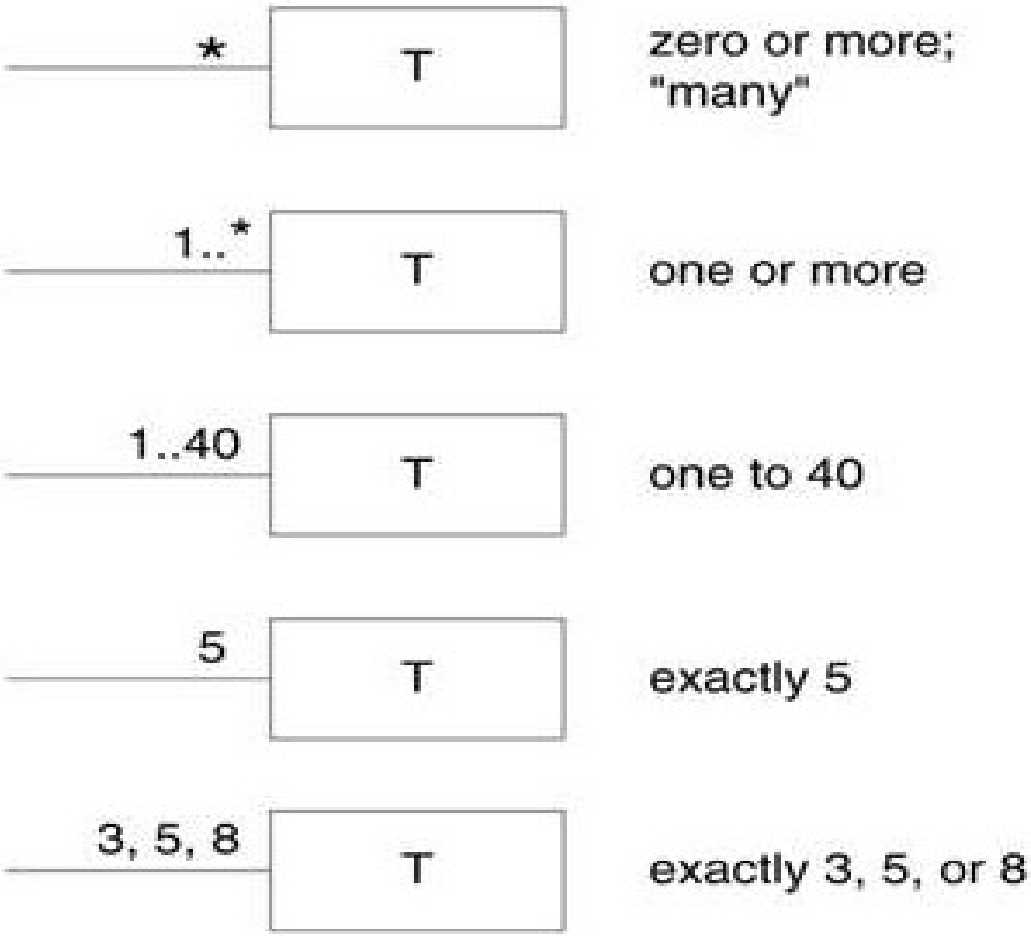
Multiplicidade

- Multiplicidade
  - Descreve a cardinalidade da relação. Especifica quantas instâncias de uma classe relacionam-se a uma única instância de uma classe associada

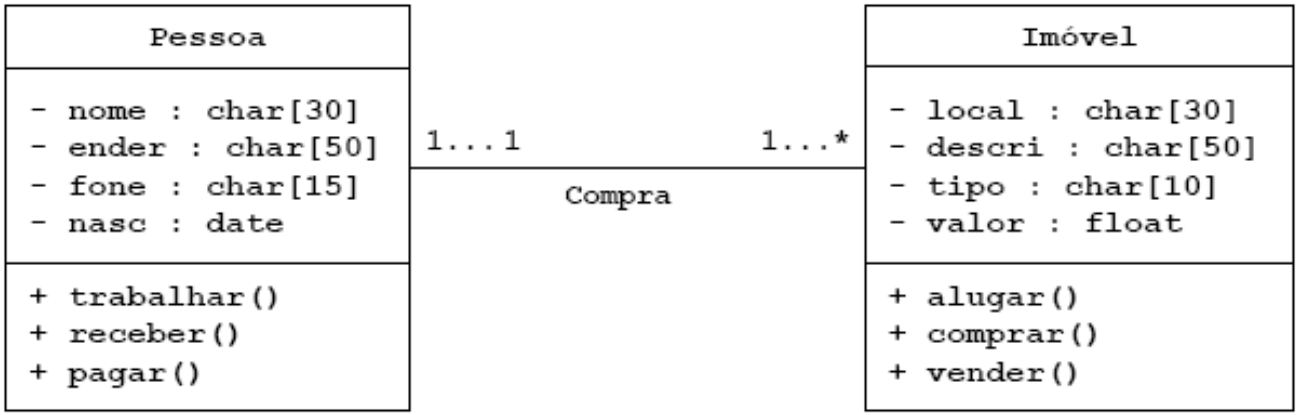




Tipos Multiplicidade



Multiplicidade

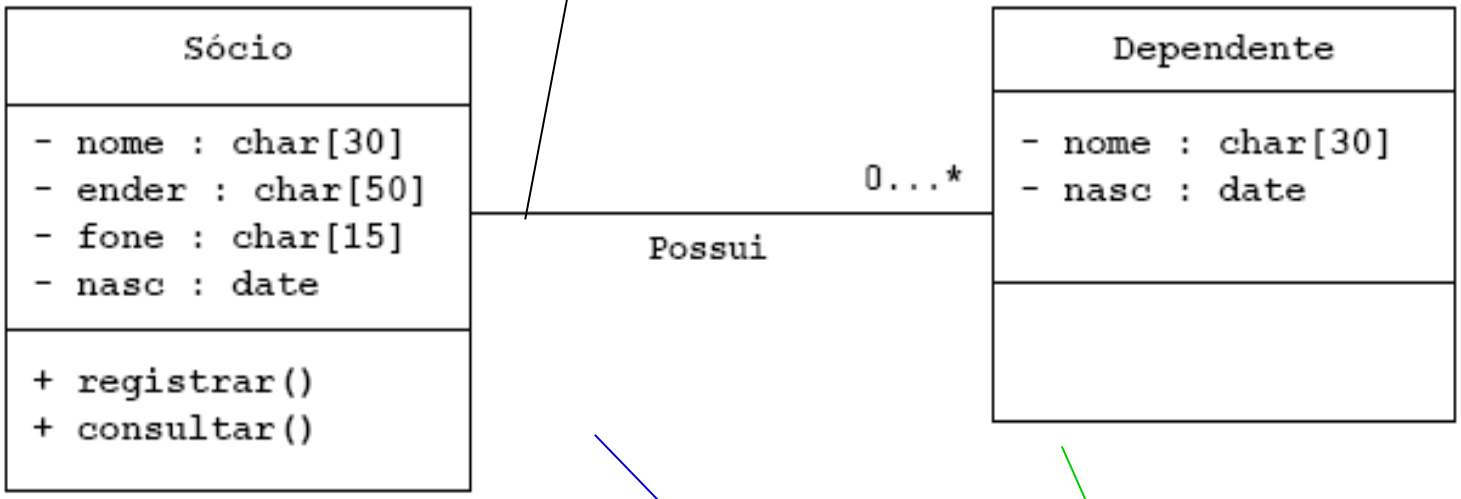


Uma Pessoa pode comprar um ou vários imóveis. Um imóvel pode ser adquirido por uma, e somente uma, pessoa

Um objeto da classe “Pessoa” pode se relacionar com várias instâncias da classe imóvel (mostrado pela multiplicidade 1...*). Por outro lado, se existir um objeto da classe “Imóvel”, obrigatoriamente, ele terá que estar relacionado a um objeto da classe “Pessoa”

Multiplicidade

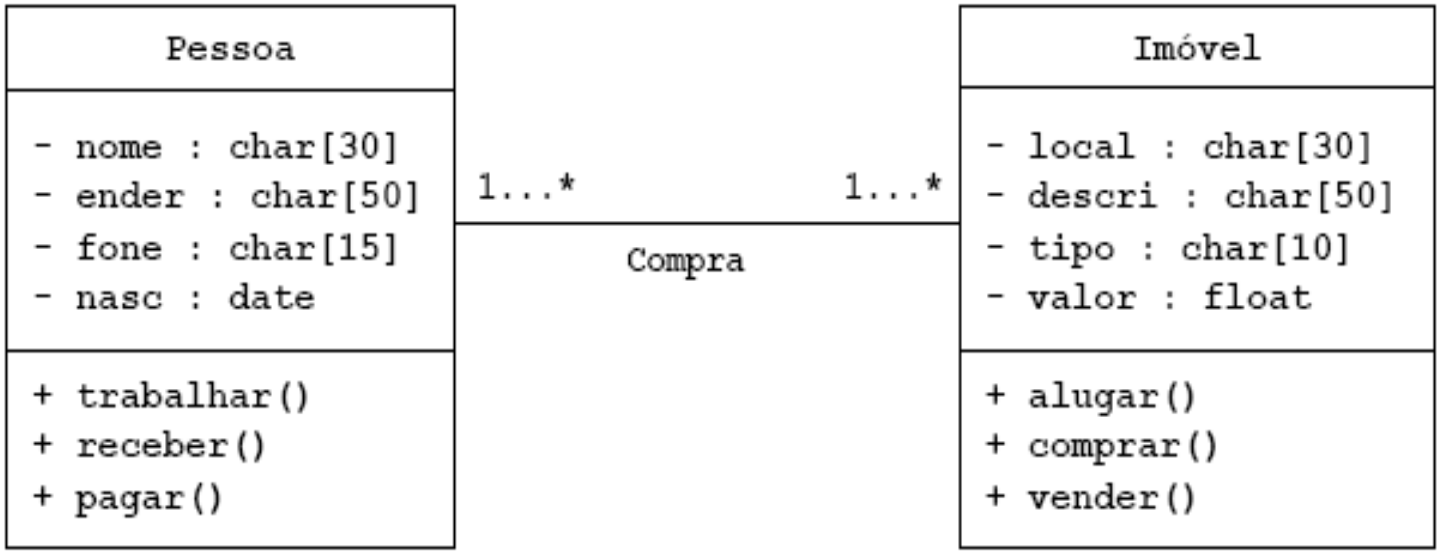
Quando omitida (não definida) entende-se a multiplicidade como sendo 1...1



Entidade “Fraca”

Um sócio pode ter 0 ou vários dependentes.
Um dependente, se existir, deve, obrigatoriamente, estar associado a um sócio

Multiplicidade



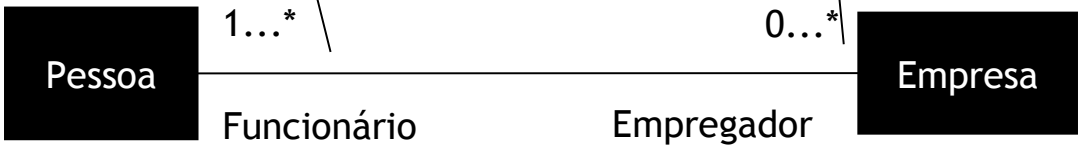
Uma Pessoa pode comprar um ou vários imóveis. Um imóvel pode ser adquirido por uma ou várias pessoas (sociedade/grupo)

A multiplicidade é definida de acordo com as necessidades do sistema ser desenvolvido

Multiplicidade

Uma empresa emprega uma ou várias pessoas

Uma Pessoa é funcionário de nenhuma ou mais empresas

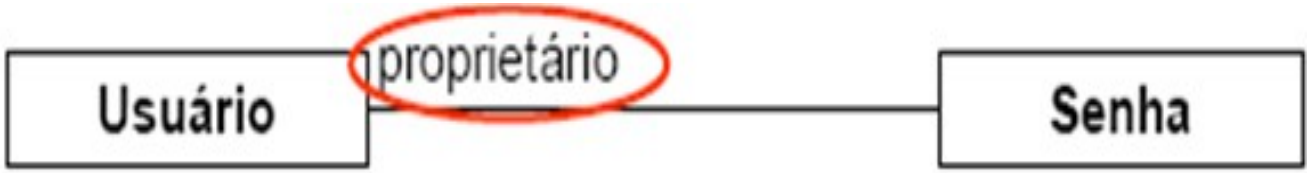


Uma empresa emprega uma ou várias pessoas

Uma Pessoa é funcionário de apenas uma empresa

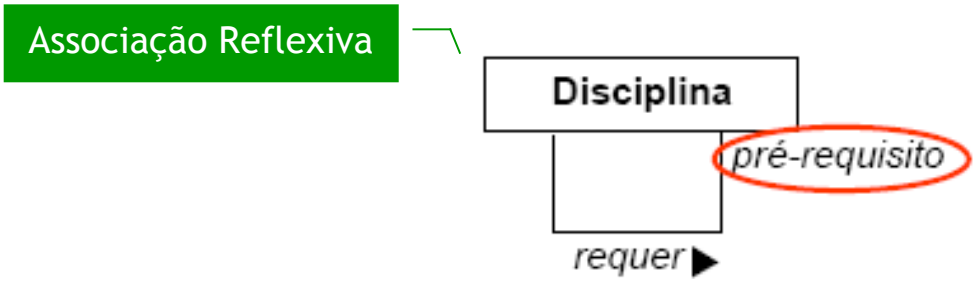
Papel

- Mostra como uma classe é vista por outra
- É um extremo de uma associação - pode ter nome
- É um uso (semântica) de uma classe em uma associação
- Uma mesma classe pode assumir vários papéis
  - Pode estar associada a várias classes



Papel

- Uso de Papéis
  - Uma única associação entre duas classes
    - Os nomes das classes servem como nomes de papéis
  - Múltiplas associações entre as mesmas classes
    - Utilizar nomes de associações ou de papéis
  - Uma associação entre objetos da mesma classe (reflexiva)
    - Utilizar nomes de papéis

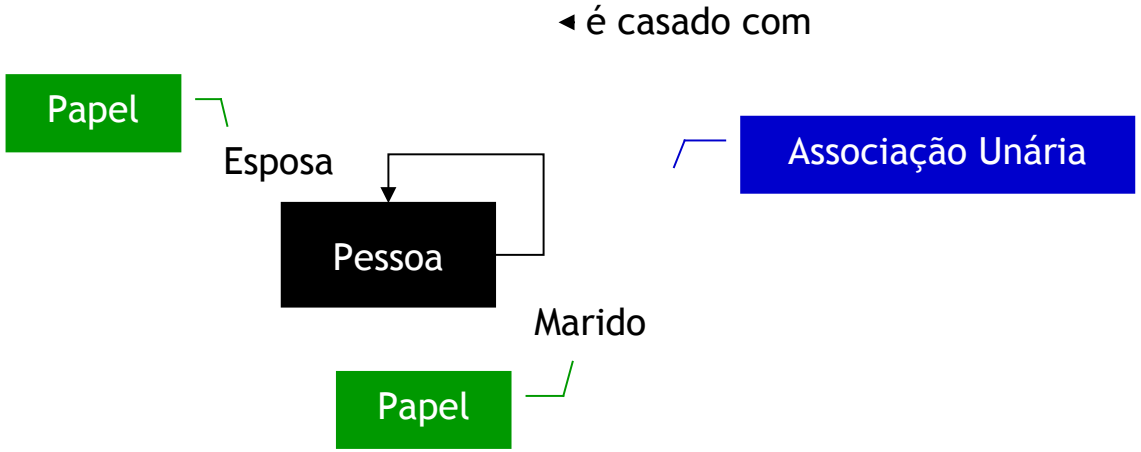


Grau

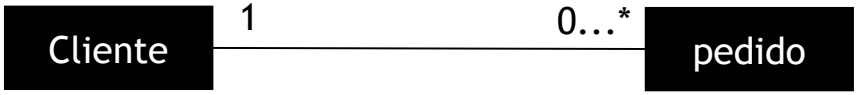
- Grau de Associação
  - É o número de papéis dos relacionamentos
    - Pode ser: unária, binária, ternária, ..., n-árias

Associação Unária

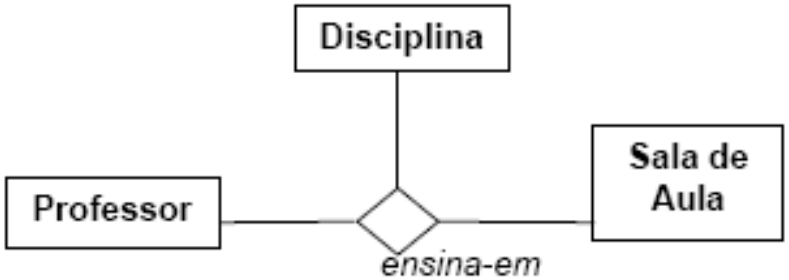
- Associação Unária
  - Associação reflexiva
  - Semanticamente representa a conexão entre dois objetos da mesma classe
  - Ocorre quando há um relacionamento de uma classe consigo própria conectando-se ambos os fins da associação a ela mesma (mas os dois fins são distintos)



Associação Binária/Ternária



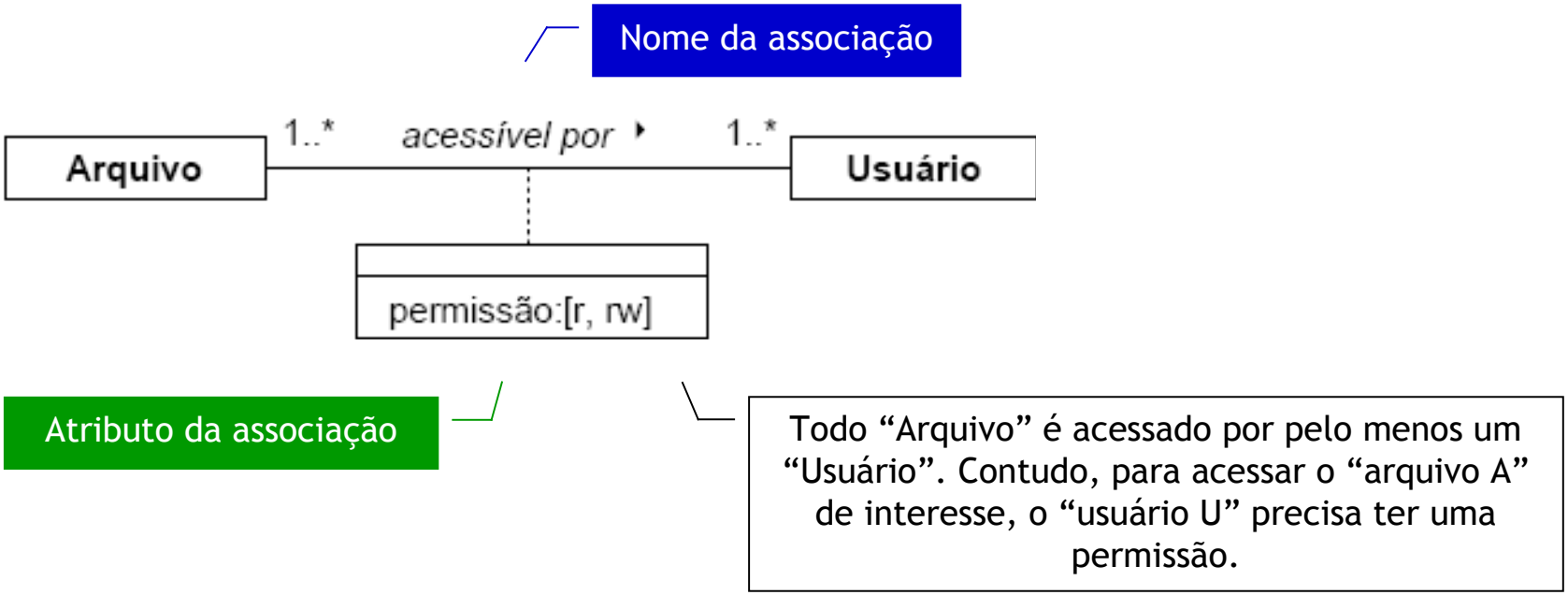
Associação Binária



Associação Ternária

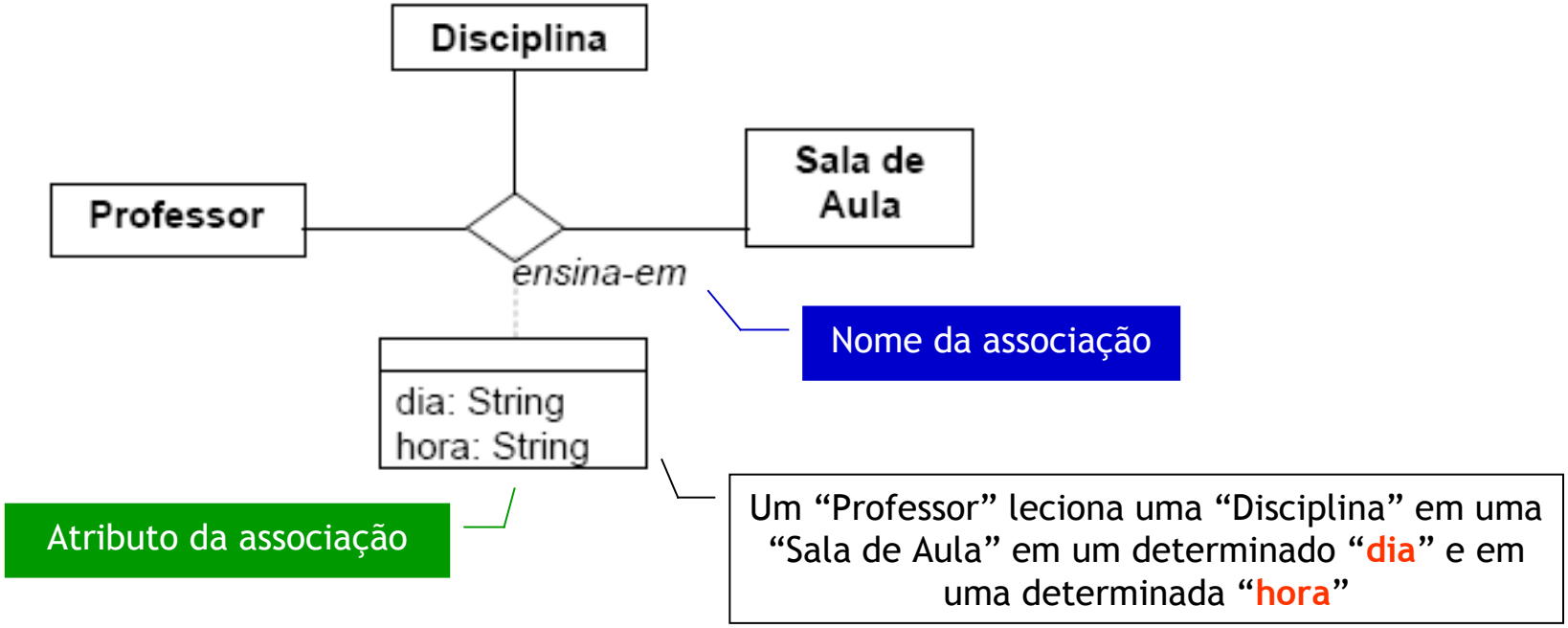
Atributo

- Atributo de Associação
  - Descreve um valor para cada relacionamento da associação



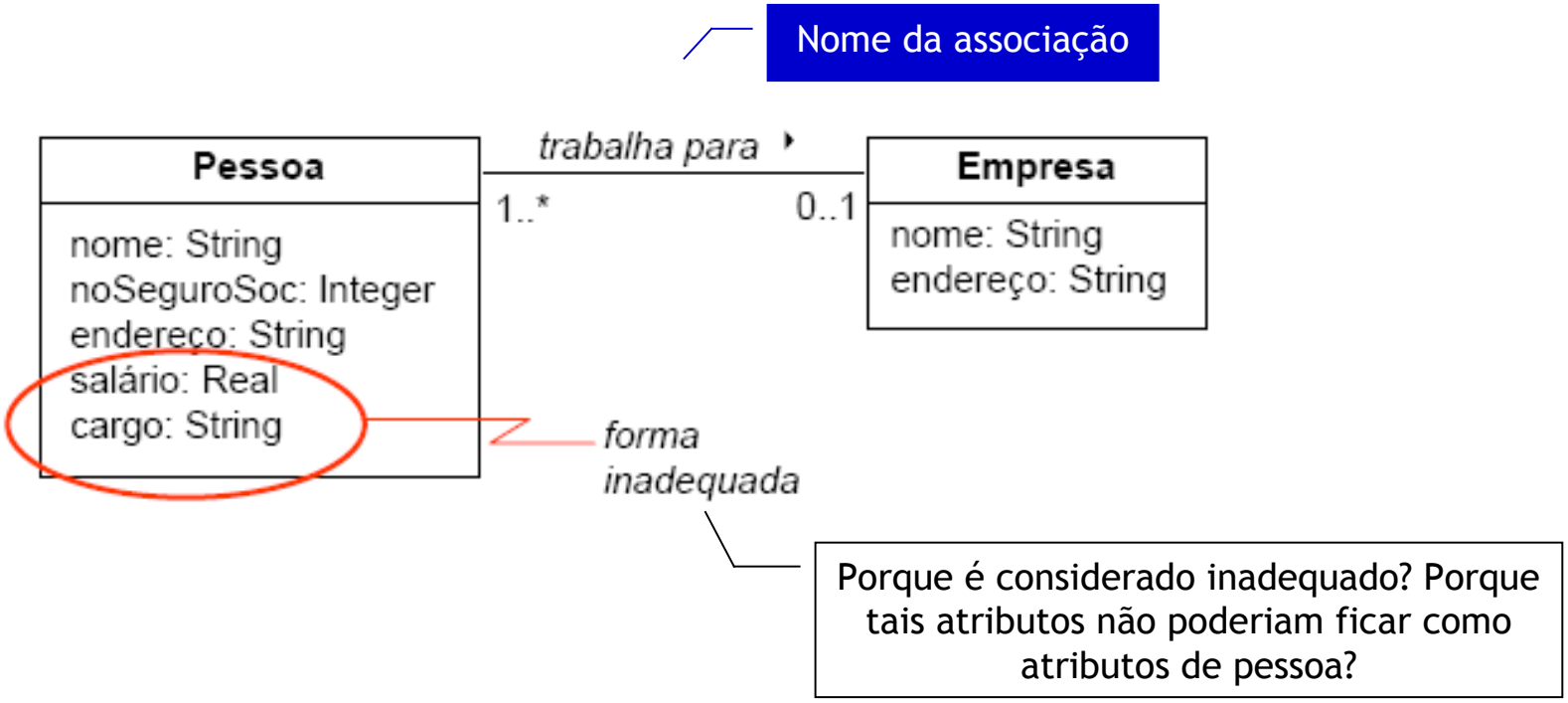
Atributo

- Atributo de Associação (Ternária)



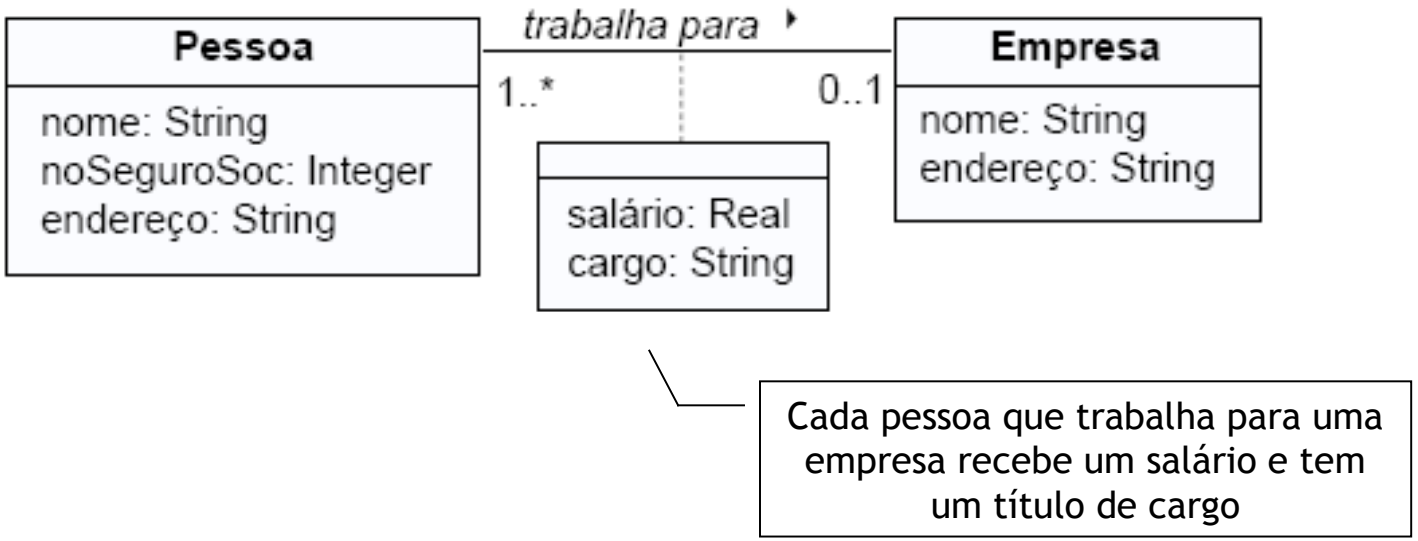
Atributo

- Atributo de Associação
  - Em associações “muitos para um” ou “um para um”, é fácil confundir atributos de associação com atributos de objetos.

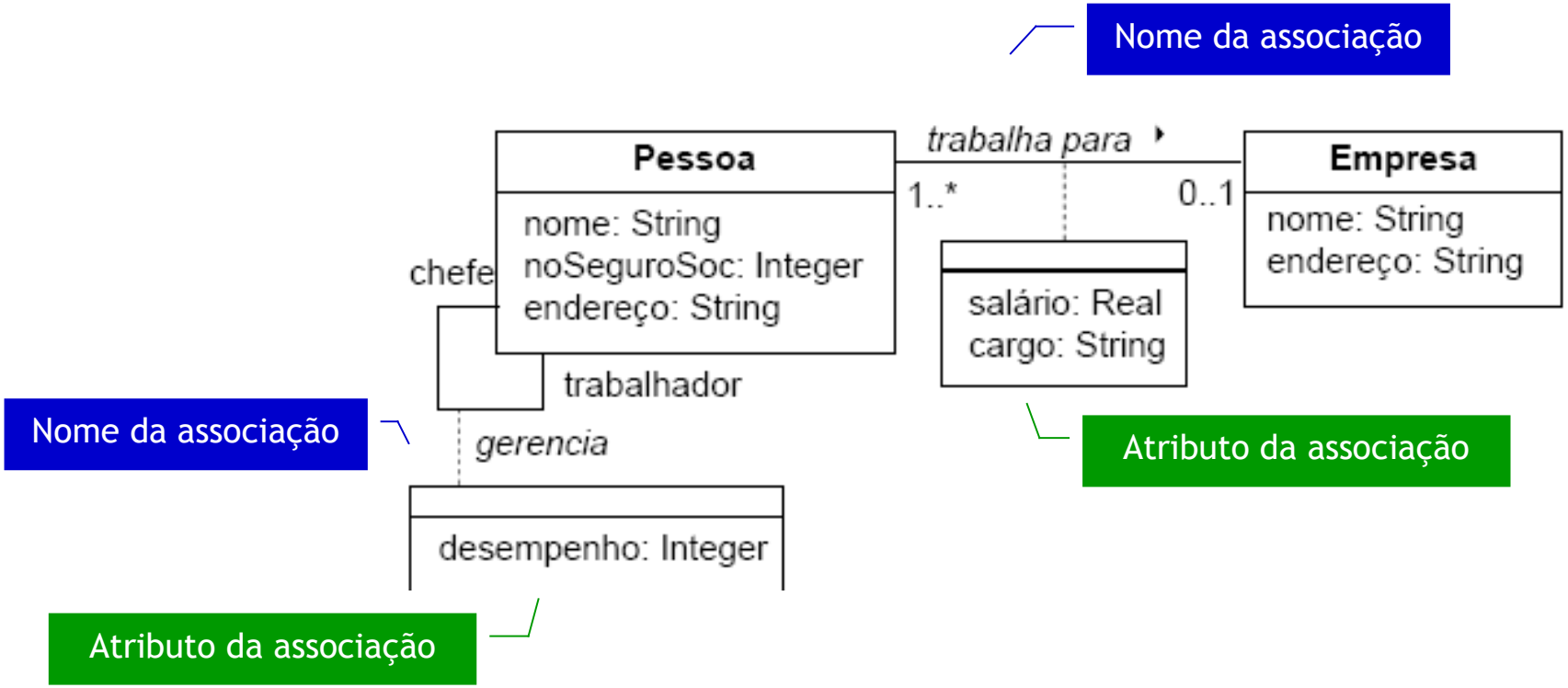


Atributo

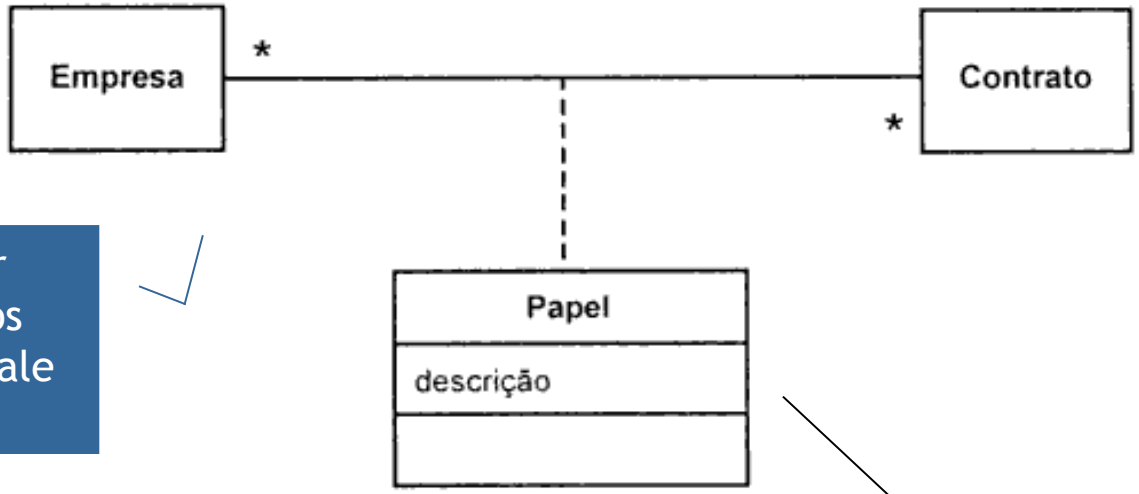
- Atributo de Associação



Atributo


Nome da associação
Pessoa
nome: String
noSeguroSoc: Integer
endereço: String
trabalha para
1..*
0..1
Empresa
nome: String
endereço: String
salário: Real
cargo: String
Atributo da associação
chefe
trabalhador
Nome da associação
gerencia
desempenho: Integer
Atributo da associação

Classe Associativa

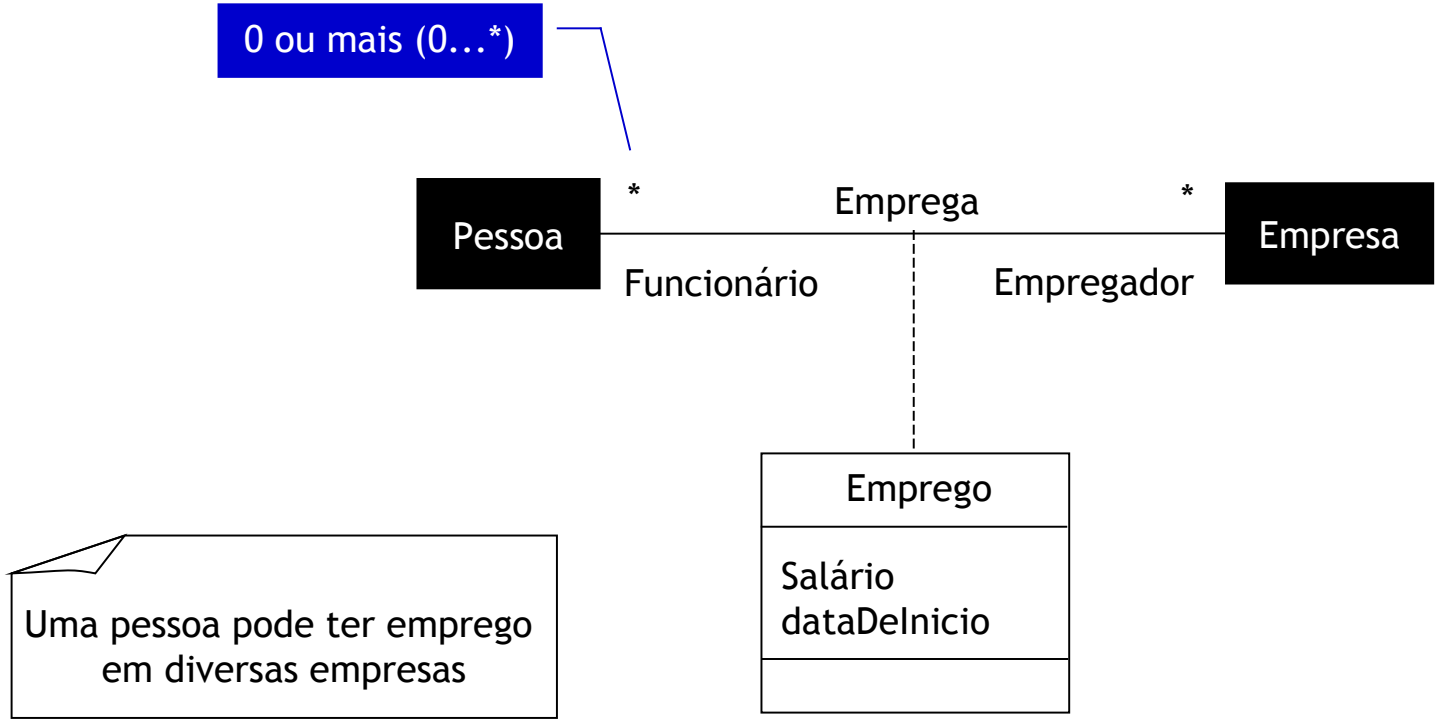


Uma empresa pode possuir nenhum ou muitos contratos associados e ela – o mesmo vale para contratos

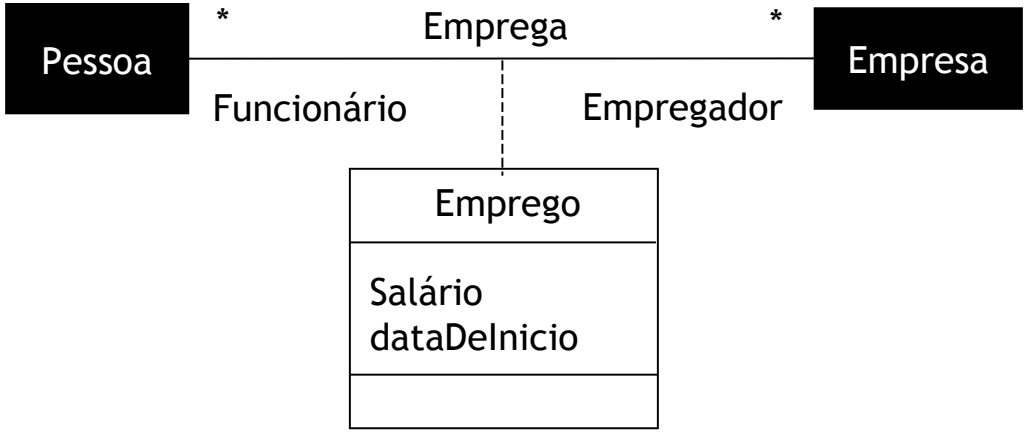
Uma classe de associação acrescenta uma restrição a mais, no sentido de que pode haver apenas uma instância da classe de associação entre quaisquer dois objetos participantes. Neste caso, toda empresa tem um papel dentro de um contrato e o intuito seria especificar que ela possui apenas um papel. Não confundir isso com a leitura da associação

Classe Associativa

- Definição
  - Uma classe associativa permite tratar uma associação em si como uma classe e modelá-la com atributos, operações e outras características.



Classe Associativa

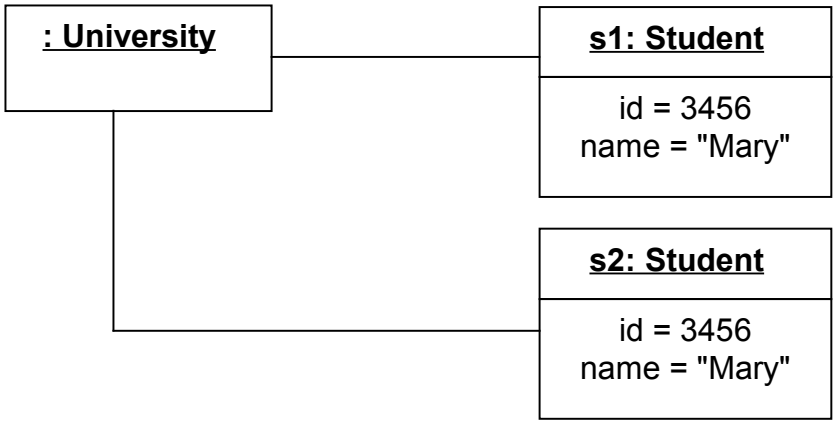


Associação Qualificada

- Definição
  - Associação que possui um “qualificador”, o qual é usado para selecionar um objeto (ou objetos) a partir de um conjunto maior de objetos – baseado na chave do qualificador
  - Associação qualificada
    - Especifica como um determinado objeto no final da associação “n” é identificado. Pode ser visto como um tipo de chave para separar todos os objetos na associação.

Associação Qualificada

(1)



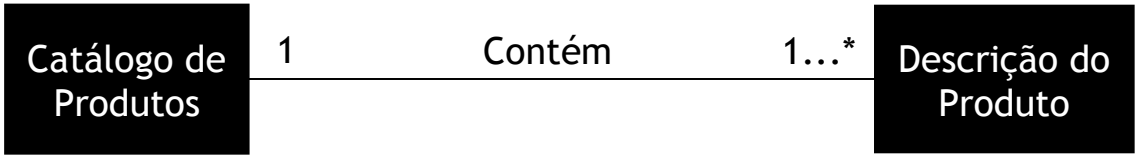
Em uma situação real, gostaríamos de garantir que cada estudante fosse unicamente identificado por um número identificador.

Associação Qualificada

(2)

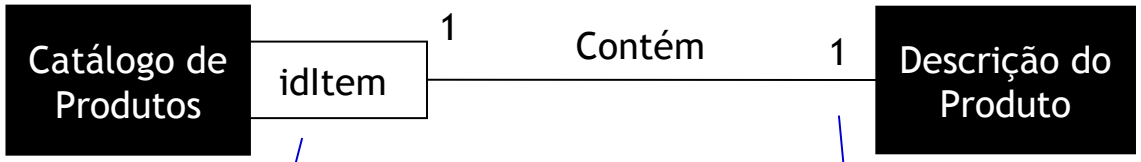
Se quisermos, no modelo, que o atributo id seja único entre todos os estudantes da universidade (ou seja, entre todos os objetos da classe Student, podemos utilizar a ideia de associação qualificada entre as classes University e Student

Associação Qualificada



Um catálogo de produtos contém muitas descrições de produto - cada uma pode ser selecionada por um identificador (idItem)

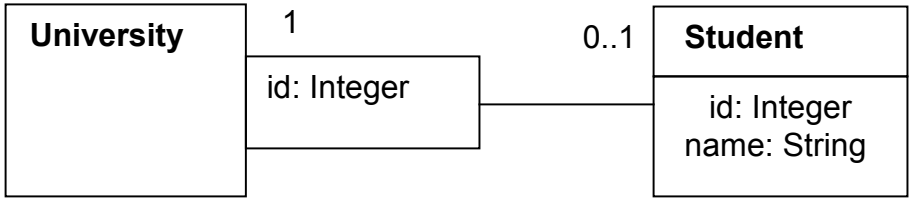
Multiplicidade Reduzida para 1

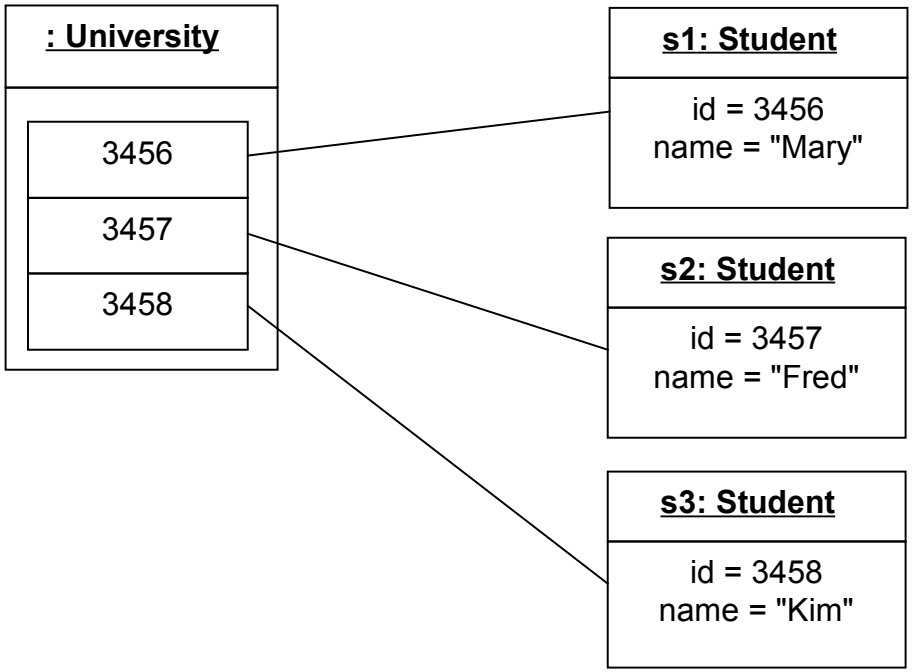


Qualificador

Um item de um catálogo de produtos possui uma única descrição

Associação Qualificada





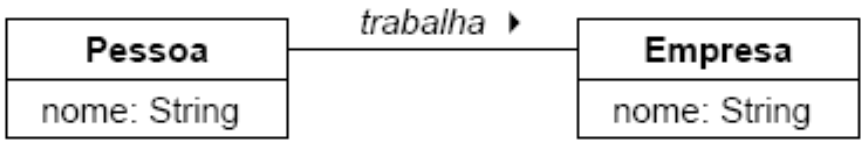
Navegação

- Associação Bidirecional:
  - É possível navegar de objetos de uma classe até objetos da outra classe (e vice-versa). Uma associação é bidirecional, sempre que não seja especificado o contrário

- Associação Navegável:
  - Partindo de um objeto em uma extremidade é possível chegar, direta e facilmente, a objetos na outra extremidade. Geralmente o objeto de origem armazena referências aos objetos de destino

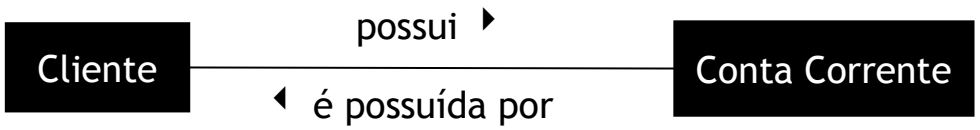
Associação Bidirecional

Uma pessoa trabalha para uma empresa



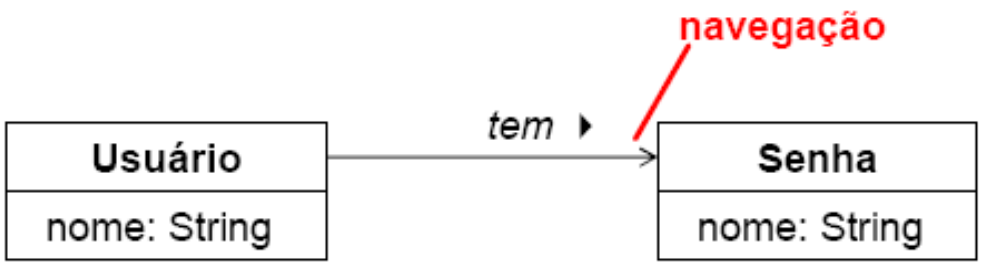
Uma empresa emprega uma pessoa

Um cliente possui uma conta corrente



Uma conta corrente pertence a (é possuída por um) cliente

Associação com Navegação



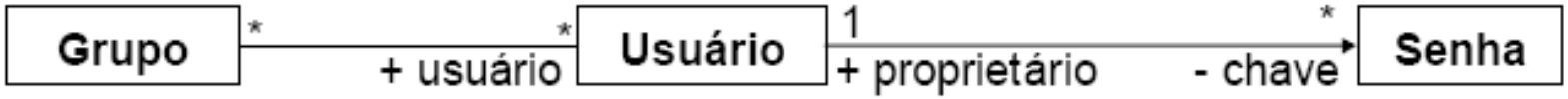
Permite ir de um usuário até sua senha, mas não permite, a partir de uma senha, identificar o usuário

Visibilidade

- Visibilidade
  - É possível limitar a visibilidade de uma associação, relativa a objetos externos à associação. Essa limitação pode ser:
  - +Pública, se não for especificado o contrário
  - -Privada
    - Os objetos existentes nessa extremidade da associação não estão acessíveis a qualquer objeto externo da associação
  - #Protegida:
    - Indica que os objetos existentes nessa extremidade da associação não estão acessíveis a qualquer objeto externo da associação, com exceção dos filhos existentes na outra extremidade

Visibilidade

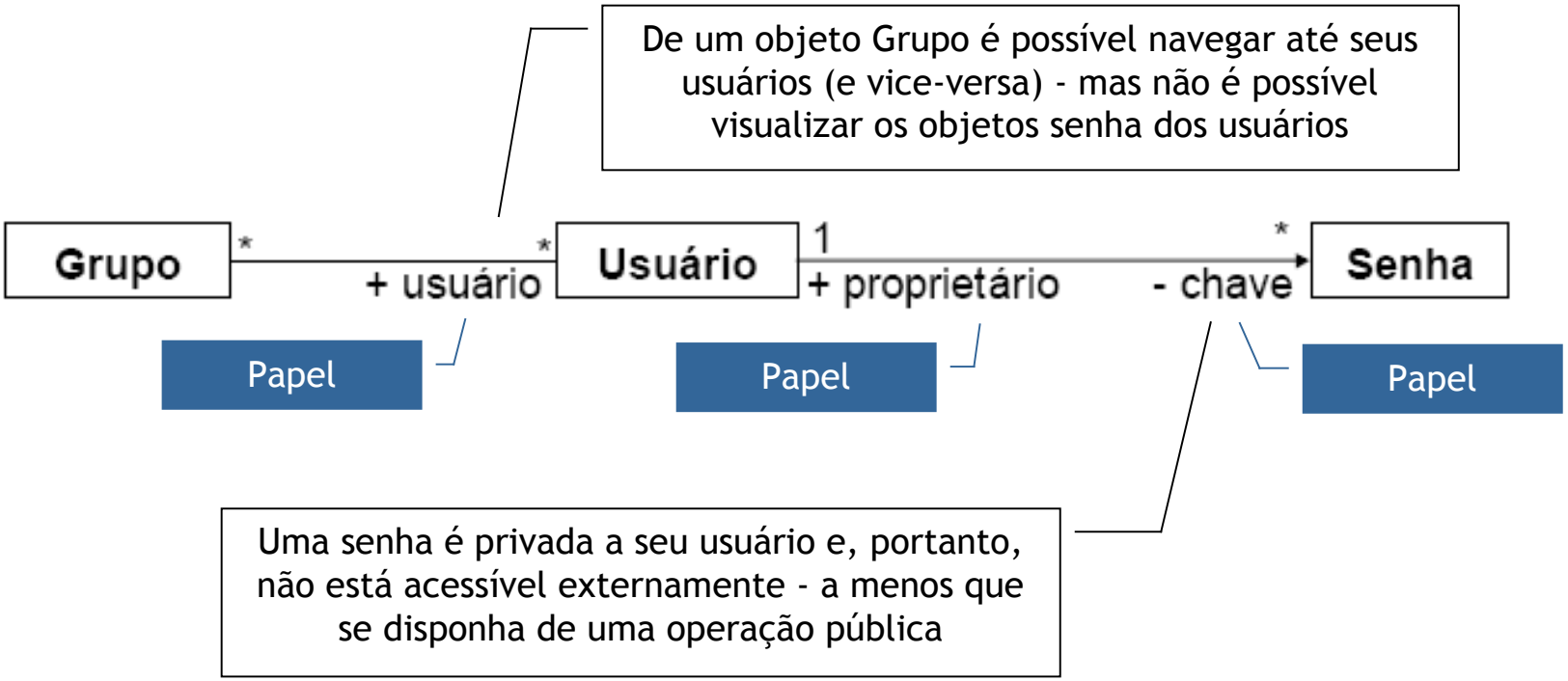
Um grupo de usuário tem vários usuários
Um usuário pode pertencer a vários grupos



Um usuário pode ter várias senhas
Uma senha somente pertence a um usuário

Navegação de Senha para
Usuário não é permitida

Visibilidade



Dependência

Dependência

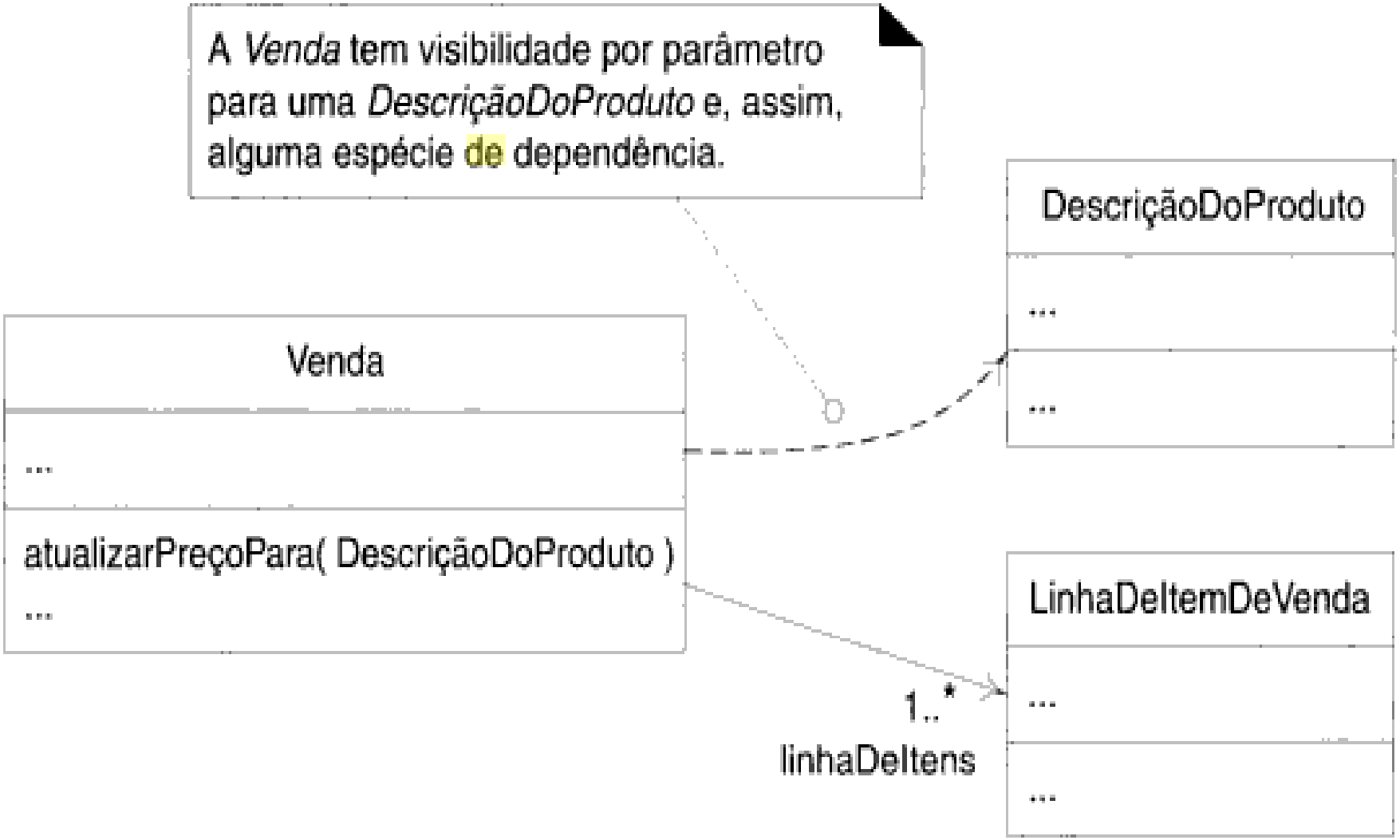
- Indica que uma classe depende dos serviços (operações) fornecidos por outra classe. Além disso, uma modificação no fornecedor pode afetar o cliente (visto que um depende dos serviços de outro).
- Tipos
  - Ter um atributo do tipo fornecedor
  - Enviar uma mensagem para um fornecedor
  - Receber um parâmetro do tipo fornecedor
  - O fornecedor é uma superclasse ou interface

  - OBS: todos poderiam ser representados com uma linha de dependência, MAS alguns desses tipos já têm linhas especiais que sugerem dependência.

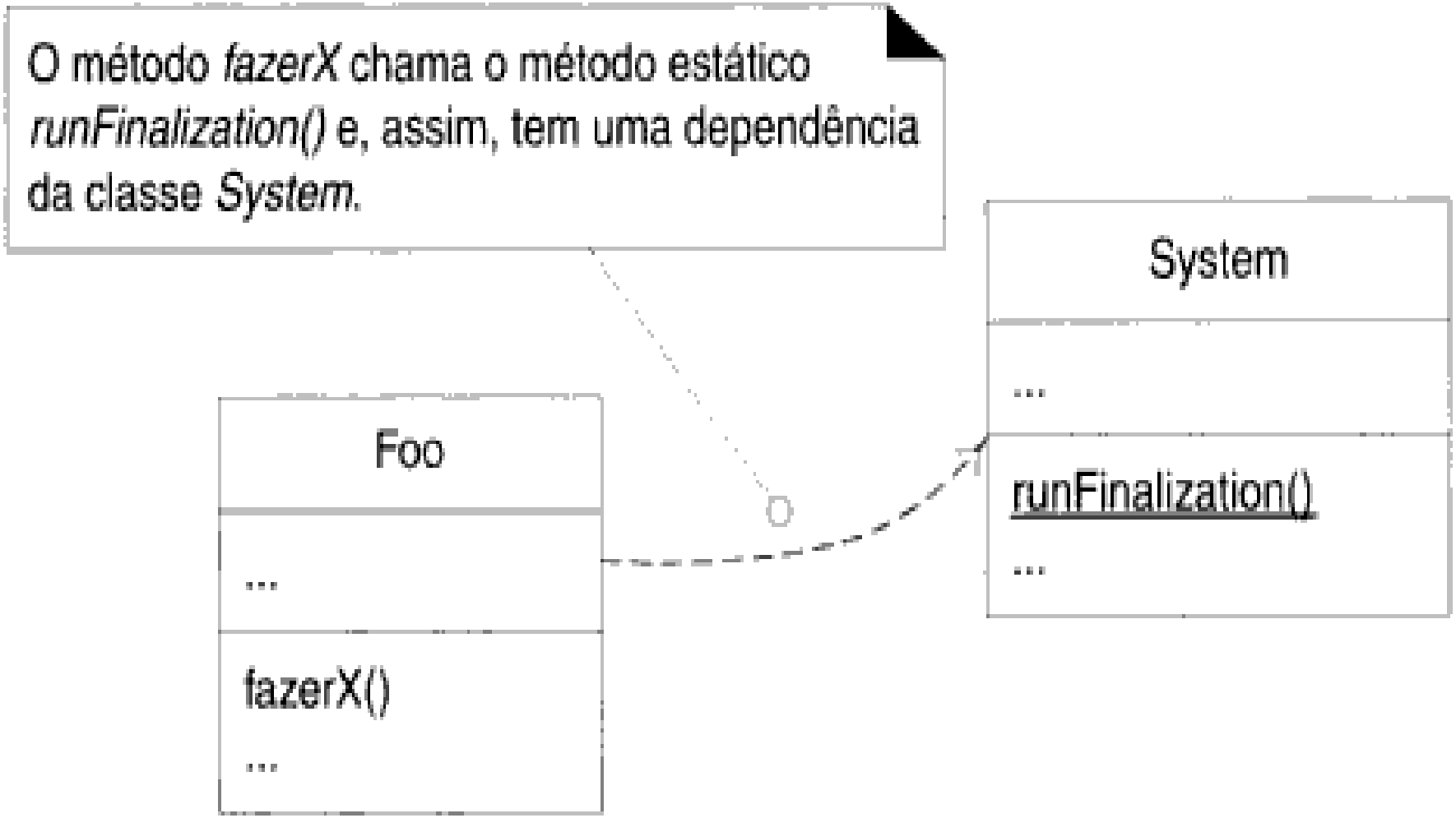
Dependência

- Detalhando “Enviar mensagem para fornecedor”
  - A visibilidade do fornecedor pode ser
    - Um atributo
    - Uma variável de parâmetro
    - Uma variável local
    - Uma variável global
    - Métodos estáticos (variáveis de classe)

Dependência



Dependência



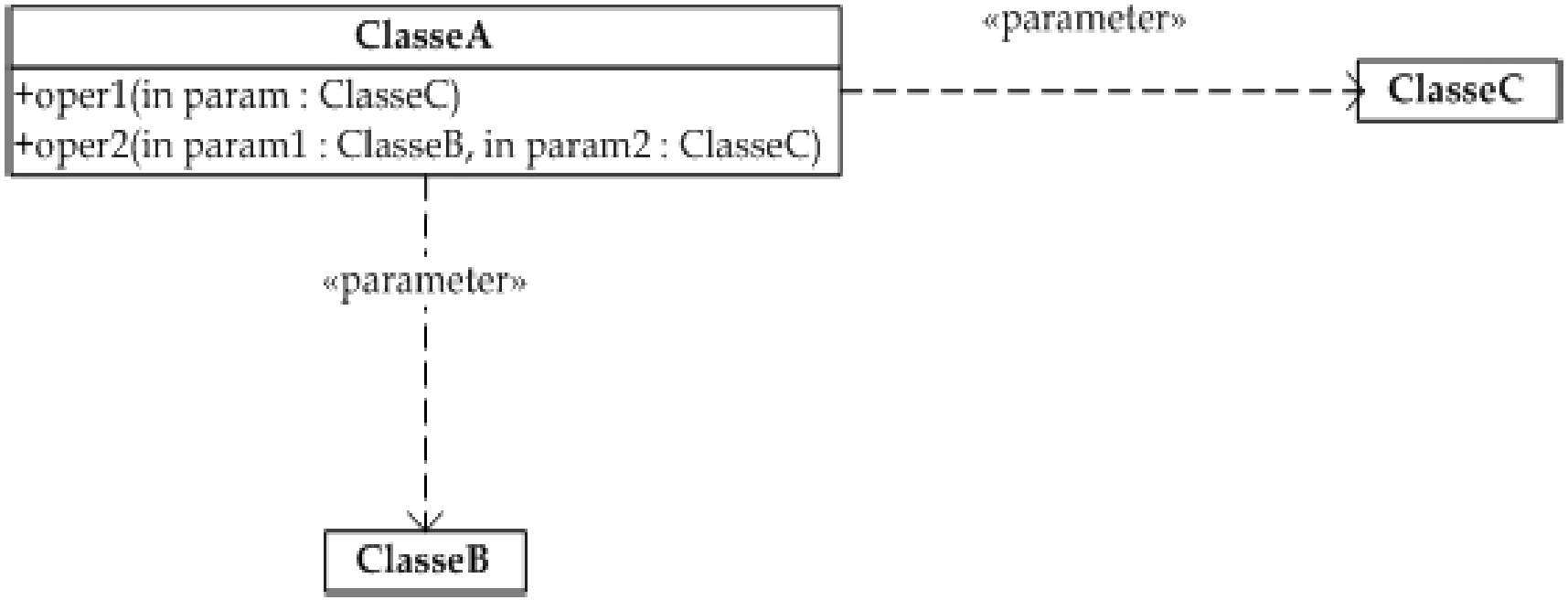
Dependência

- Uma das dependências mais comuns (não estruturais), está na dependência por parâmetro, um objeto recebe outro como parâmetro em um método.

- Na dependência por variável local, um objeto recebe outro como retorno de um método, ou possui uma referência para o outro objeto como uma variável local em algum método.

Dependência Não Estrutural

- São representadas na UML por uma linha tracejada direcionada e ligando as classes envolvidas. A direção é da classe dependente (cliente) para a classe da qual ela depende (fornecedora).



Dependência

- Observação Importante
  - Durante o projeto de classes, é necessário avaliar, para cada associação existente (dependência estrutural), se é possível transformá-la em uma dependência não estrutural.

  - Objetivo
    - Aumentar o encapsulamento de cada classe e diminuir a acoplamento entre as classes. A dependência por atributo é a forma mais forte de dependência.
    - OBS: Quanto menos dependências por atributo houver no modelo de classes, maior é o encapsulamento e menor o acoplamento.

Relacionamento Entre Classes

- Associação
  - A classe dependente possui um atributo que é uma referência para a outra classe. Também chamada de dependência por atributo (ou estrutural)

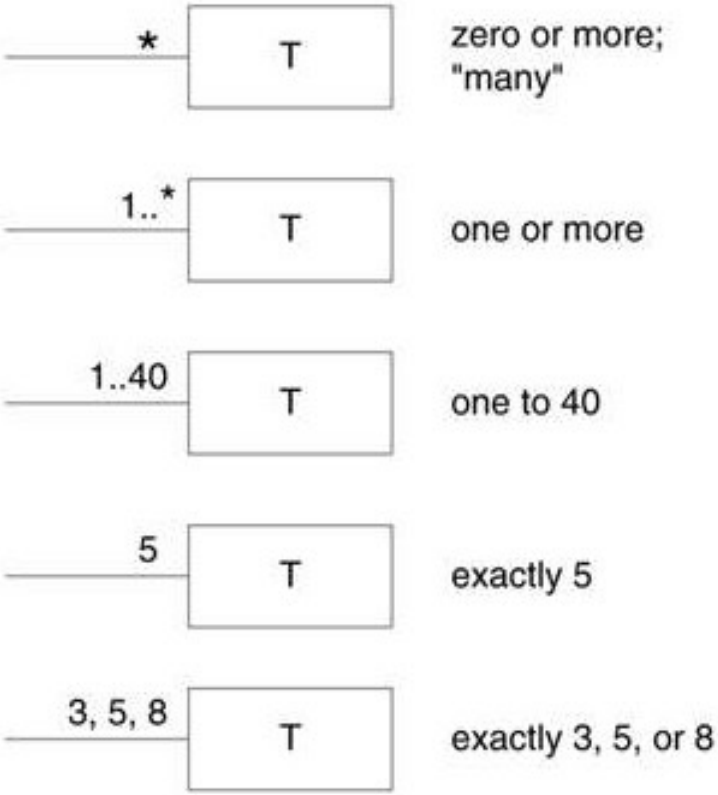
- Dependência
  - Indica que uma classe depende dos serviços (operações) fornecidos por outra classe.

Implementação - Associação

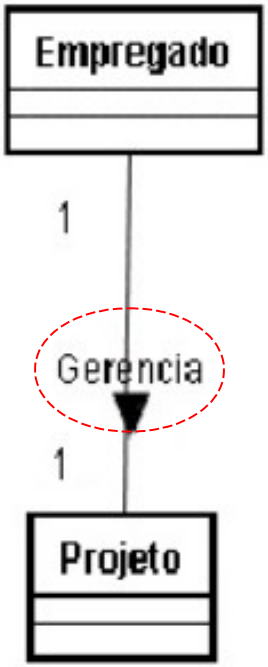
Associação

- Associações
  - Bidirecional
  - Unidirecional

Associação


*
T
zero or more;
"many"
1..*
T
one or more
1..40
T
one to 40
5
T
exactly 5
3, 5, 8
T
exactly 3, 5, or 8

Associação

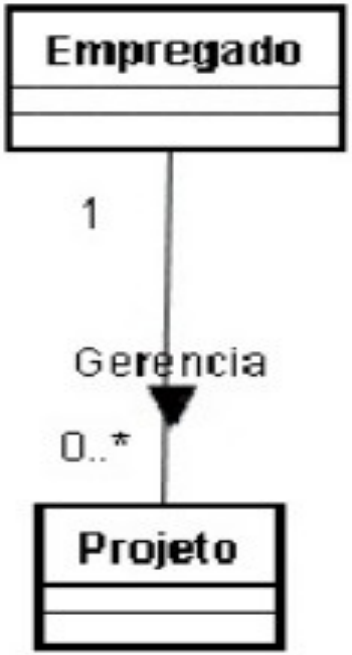



```
public class Empregado {

    private Projeto projeto;
}
```

Sentido de Leitura

```
public class Projeto {

    private Empregado empregado;
}
```

Associação



Exemplo de associação

Observe que um Empregado pode gerenciar nenhum ou vários projetos. Estamos lidando aqui com o conceito de coleções

```
import java.util.ArrayList;

public class Empregado {

    private ArrayList<Projeto> projetos = new ArrayList<Projeto>();
}
```

Abre parênteses -> Coleções

Coleções

- Estruturas de dados pre-definidas

- Oferecem metodos eficientes que:
  - Organizam,
  - Armazenam e,
  - Recuperam os dados.

  Ex: ArrayList (Classe genérica)

ArrayList

Métodos comuns:

- **add** - adiciona elemento ao fim do ArrayList
- **get** – retorna o elemento no indice especificado
- **contains** – retorna true se ArrayList contiver o elemento especificado
- **clear** – remove todos os elementos do ArrayList
- **remove** – remove o elemento no indice especificado
- **size** – retorna o numero de elementos armazenados no ArrayList

ArrayList

Exemplo Bloco de Notas

Contrua uma classe capaz de armazenar anotações, exibir e remover uma determinada anotação.

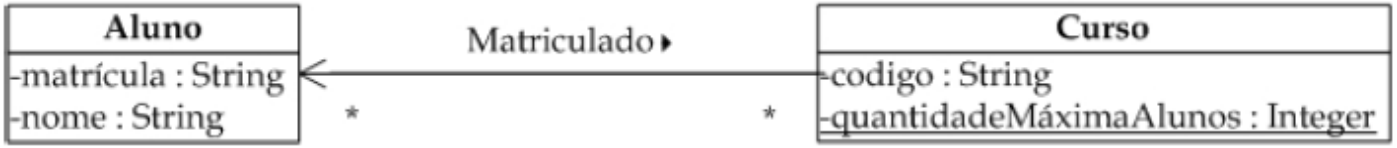
Coleções <- Fecha parênteses

Associações

- Associações
  - Associações podem ser bidirecionais ou unidirecionais.
  - Bidirecional
    - Indica que há um conhecimento mútuo entre os objetos associados.
  - Unidirecional
    - Indica que apenas um dos extremos da associação tem ciência da existência da mesma. Representada através da adição de um sentido à seta da associação.

Associações

- Associações
  - A escolha da navegabilidade de uma associação pode ser feita através do estudo dos diagramas de interação. O sentido de envio das mensagens entre objetos influencia na necessidade ou não de navegabilidade em cada um dos sentidos.

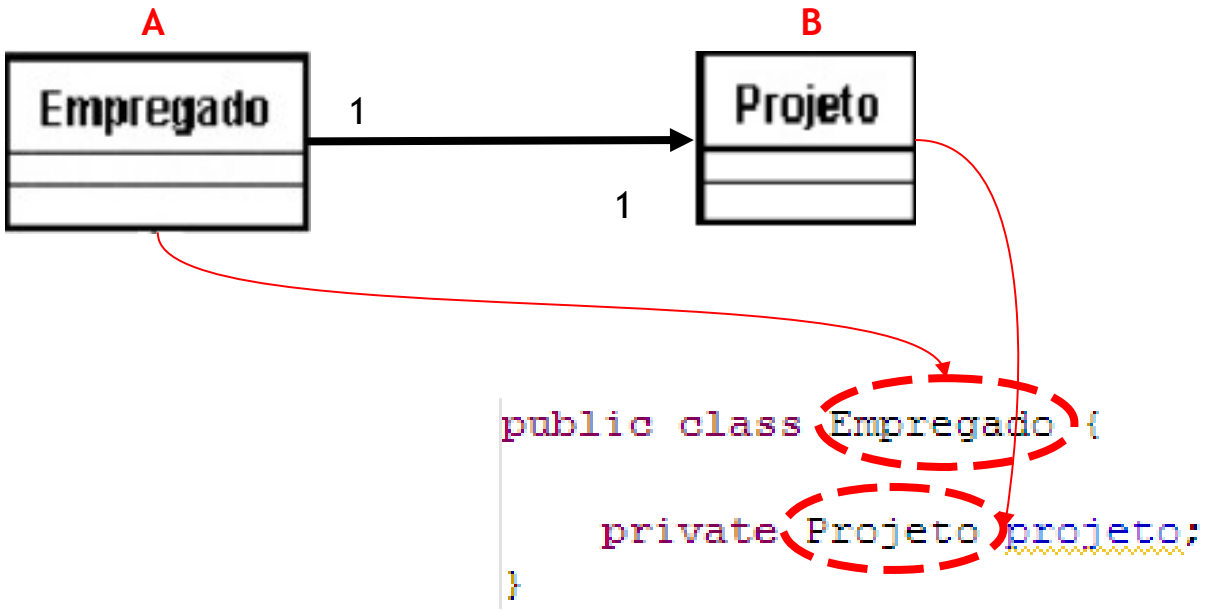


Implementação de Associações

- Há três casos
  - Em função da conectividade: 1:1, 1:N e N:M
  - Para uma associação 1:X entre duas classes A e B:
    - Se a navegabilidade é unidirecional no sentido de A para B, é definido um atributo do tipo B na classe A.Se a navegabilidade é bidirecional, podemos aplicar o procedimento acima para as duas classes.

Relacionamento Unidirecional

Unidirecional Para 1



Unidirecional Para 1

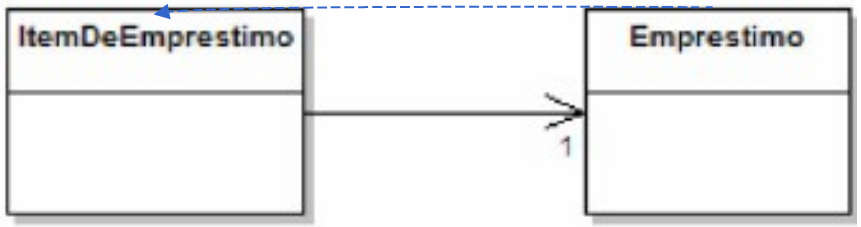
Unidirecional

Como a associação 1 é obrigatória para o objeto origem,construtor da classe deve ter como parâmetro o elemento a ser associado, para que, desde o momento da criação, todas as instâncias da classe na origem da associação estejam consistentes.

Unidirecional Para 1

Construtor de ItemDeEmprestimo

```
class itemDeEmprestimo
{
        private Emprestimo emprestimo;

        public ItemDeEmprestimo(Emprestimo emprestimo)
        {
                this.associaEmprestimo(emprestimo)
        }

        public void associaEmprestimo(Emprestimo emprestimo)
        {
                This.emprestimo = emprestimo;
        }

        public Emprestimo getEmprestimo()
        {
                return emprestimo;
        }
}
```

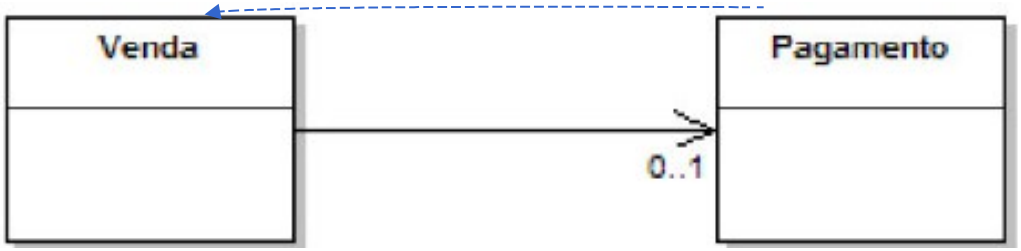


Unidirecional Para 0:1

Não é necessário passar um objeto como parâmetro para o método construtor, pois a associação 0..1 não é obrigatória

Unidirecional Para 0:1

```
class Venda
{
    private Pagamento pagamento;

    public Venda() {}

    public void associaPagamento(Pagamento pagamento)
    {
            this.pagamento = pagamento;
    }

    public void desassociaPagamento()
    {
            This.pagamento = null;
    }

    public Pagamento getPagamento()
    {
            return pagamento;
    }
}
```

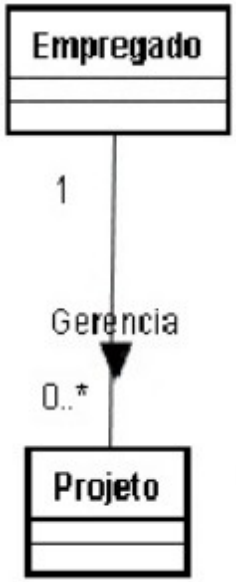


Unidirecional Para 0:1

```
public class Professor {
  private GradeDisciplinas grade;
  ...
}
```

Unidirecional Para *

- Para uma associação 1:N entre duas classes A e B:
  - São utilizados atributos cujos tipos representam coleções de elementos.
  - 1:N: essas coleções são codificadas na classe de multiplicidade 1



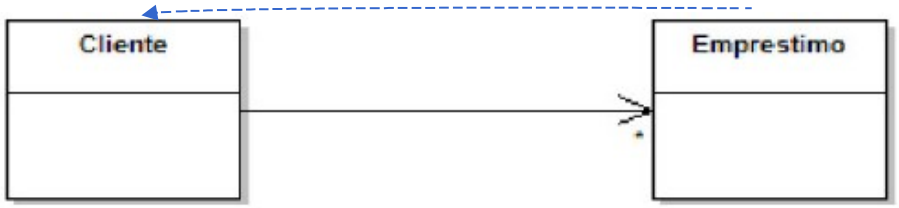

```
import java.util.Set;

public class Empregado {

    private Set<Projeto> projetos;
}
```

Unidirecional Para *

A implementação de uma associação desse tipo constitui-se pela implementação de uma variável do tipo Conjunto, no código Java da classe Cliente.

Unidirecional Para *

```
class Cliente {
        private Set emprestimos = new HashSet();
        public Cliente () { }
        public void adicionaEmprestimo(Emprestimo emprestimo)
        {
                this.emprestimos.add(emprestimo);
        }
        public void removeEmprestimo(Emprestimo emprestimo)
        {
                this.emprestimos.remove(emprestimo);
        }
        public Set getEmprestimos ()
         {
                return Collections.unmodifiableSet(emprestimos);
        }
}
```



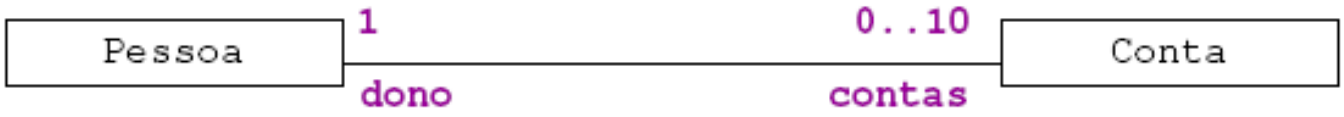
Unidirecional Para *

```
public class Aluno {
  private Set<Participacao> participacoes;
  ...
```

Unidirecional Para *

```
public class Aluno {
    private Set<Participacao> participacoes;
    ...

    public boolean adicionarParticipacao(Participacao p){
        ...
    }

    public boolean removerParticipacao(Participacao p) {
        ...
    }
}
```

Unidirecional – Valores Específicos




```
public class Pessoa {
    int numProxConta=0;
    static final int numMaxConta=10;
    Conta[] conta;
    ...
    Pessoa() {
        conta = new Conta[numMaxConta];
    }
    void associarConta(Conta conta){
        if (numProxConta<numMaxConta)
            this.conta[numProxConta++] = conta;
        else
            System.out.println("Número máximo atingido!");
    }
}
```

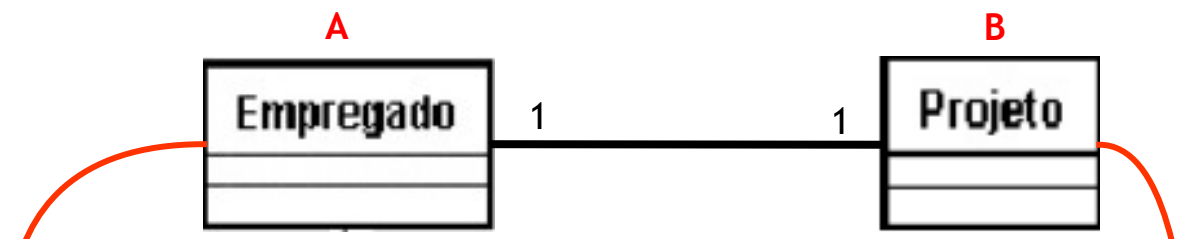
```
public class Conta{
    Pessoa dono;
    ...
    Conta(Pessoa dono){
        this.dono = dono;
    }
}
```

Relacionamento Bidirecional

Associação Bidirecional

- Associação bidirecional
  - Deve ser implementada em ambas as classes.
  - Independente de a multiplicidade ser para 1, para 0..1 ou para *

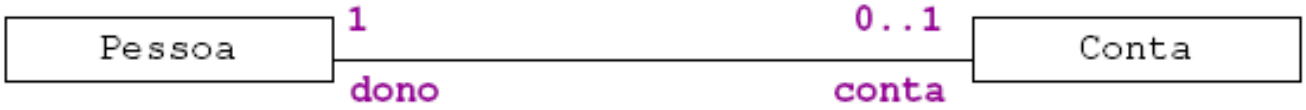
Associação Bidirecional 1:1




```
public class Empregado {

    private Projeto projeto;
}
```

```
public class Projeto {

    private Empregado empregado;
}
```

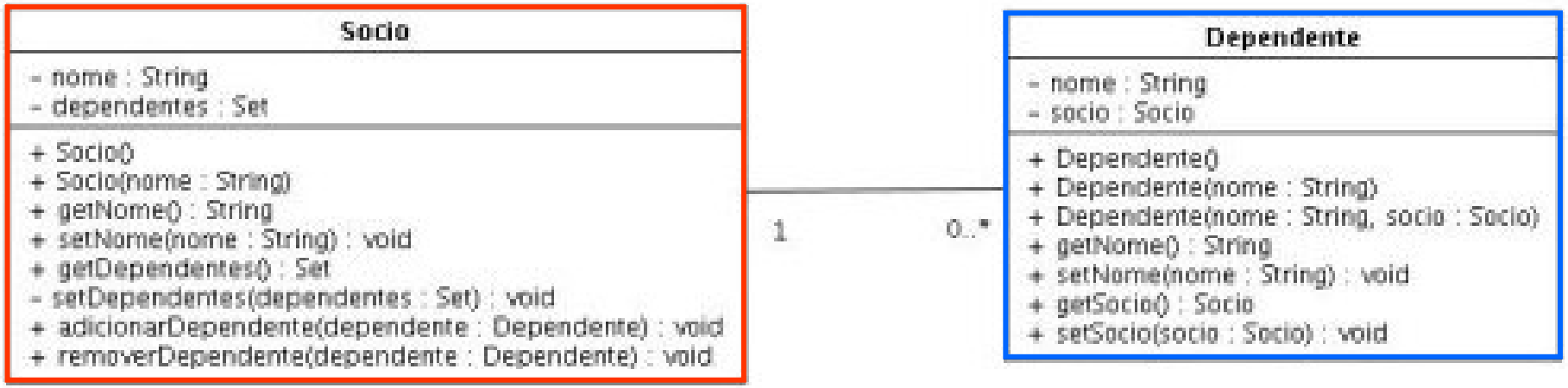
Bidirecional Para 0:1




```
public class Pessoa {
   Conta conta;
   ...
   Pessoa(){
        conta = null;
         ...
   }
   void associarConta(Conta conta) {
        this.conta = conta;
   }
}
```

```
public class Conta{
   Pessoa dono;
   ...
   Conta(Pessoa dono){
        this.dono = dono;
         ...
   }
}
```

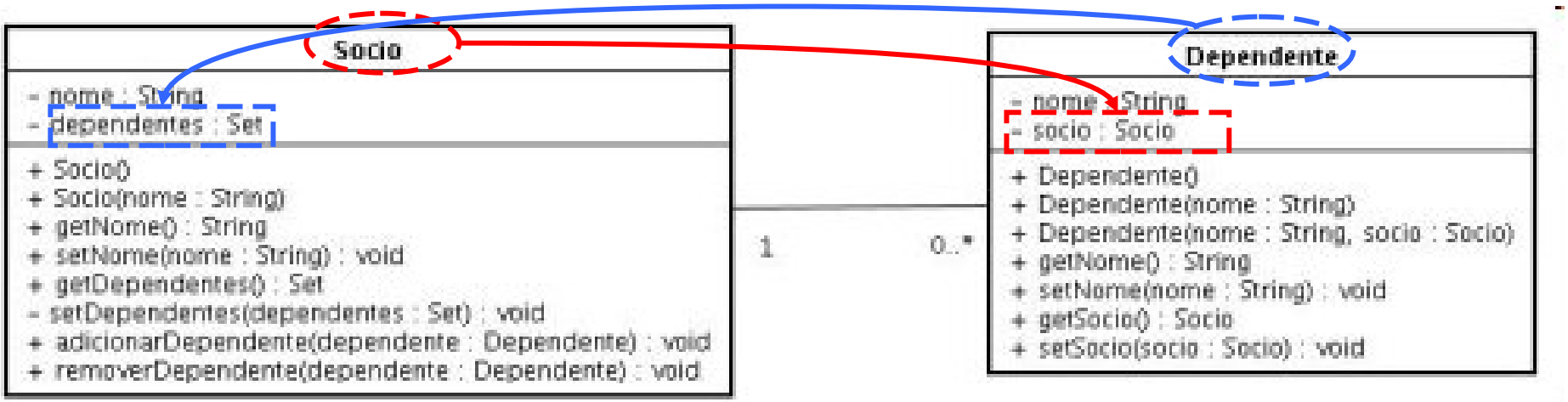
Associação Bidirecional 1:N




```
public class Socio {
        private String nome;
        private Set dependentes;
```

```
public class Dependente {
        private String nome;
        private Socio socio;
```

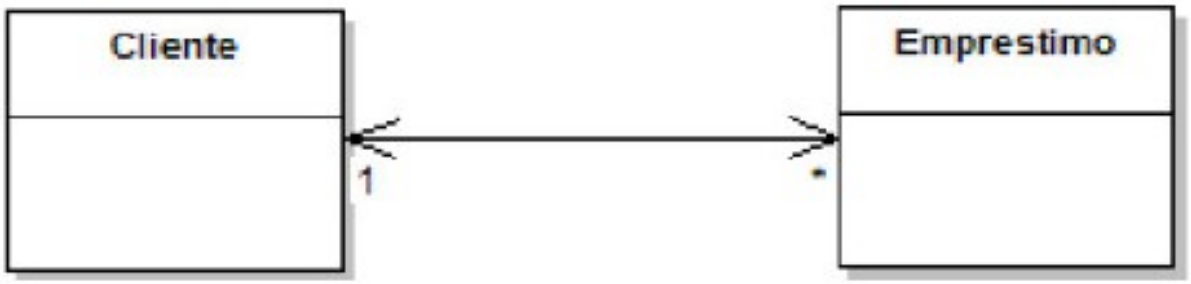
Implementação de Associações




```
public class Socio {
        private String nome;
        private Set dependentes;
```

```
public class Dependente {
        private String nome;
        private Socio socio;
```

Associação Bidirecional 1:N

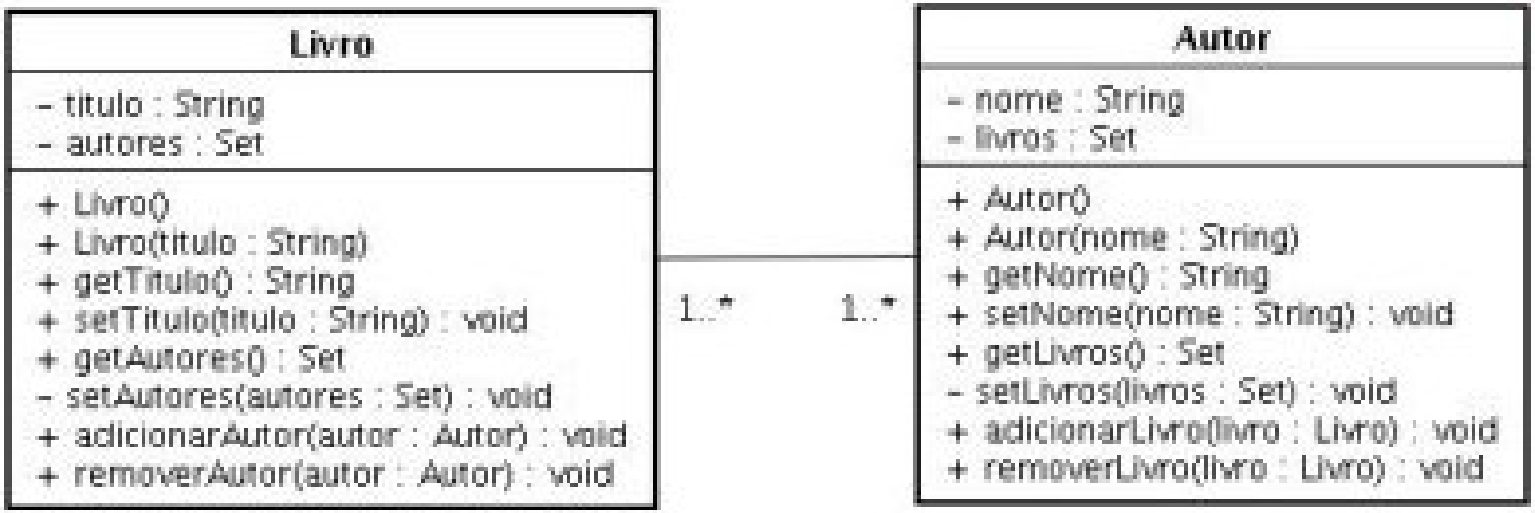


Associação Bidirecional 1:N

```
class Cliente
{
        private Set emprestimos = new HashSet();

        public Cliente () {}
```

```
class Emprestimo
{
        private Cliente cliente;

        public Emprestimo(Cliente cliente)
        {
                this.associaCliente(cliente);
        }
```

Associação Bidirecional N:N



- Implementar N:M
  - Insira, em ambas as classes, um atributo do tipo coleção
  - Insira os métodos get e set correspondentes ao atributo coleção;
  - Insira métodos para adicionar e remover objetos na coleção

Implementação de Associações

```
public class Livro {

private String titulo;
private Set autores;

...
}
```

1

```
public void setAutores(Set autores) {
...
}

public Set getAutores() {
...
}
```

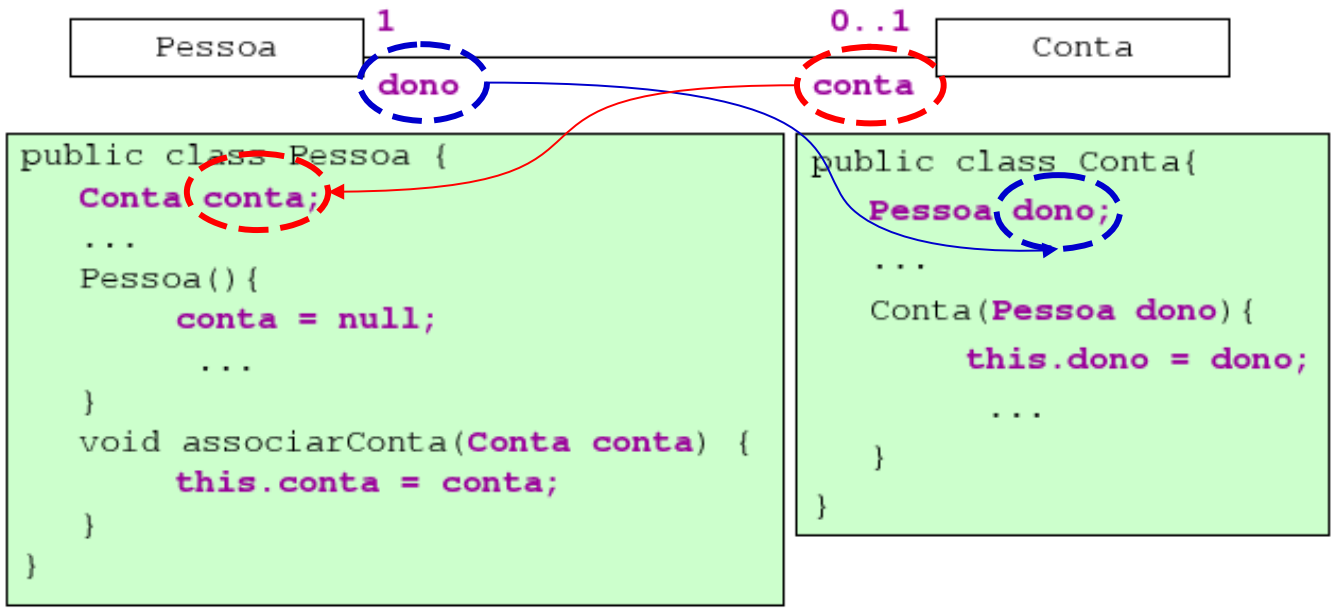
2

```
public void adicionarAutor(Autor autor){
...
}

public void removerAutor(Autor autor){
...
}
```

3

Associação - Continuação

Papéis

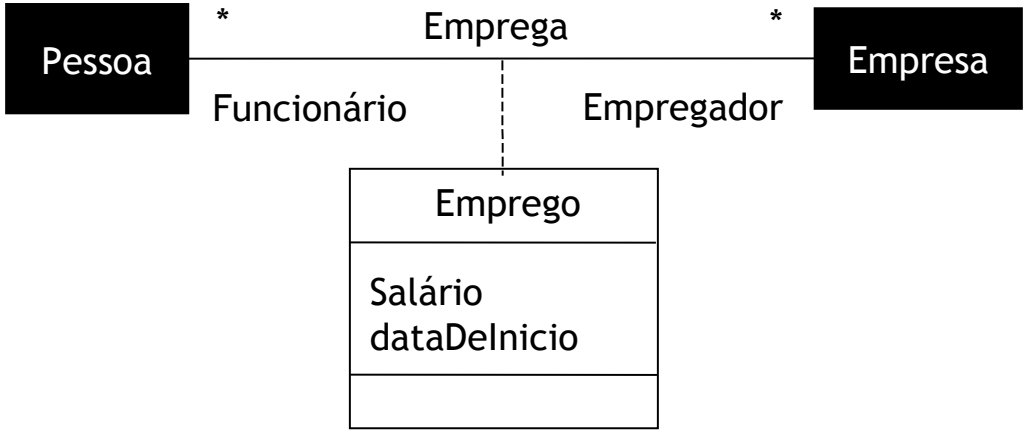

Pessoa
1
dono
0..1
conta
Conta
public class Pessoa {
Conta conta;
...
Pessoa(){
conta = null;
...
}
void associarConta(Conta conta) {
this.conta = conta;
}
}
public class Conta{
Pessoa dono;
...
Conta(Pessoa dono){
this.dono = dono;
...
}
}

Papéis

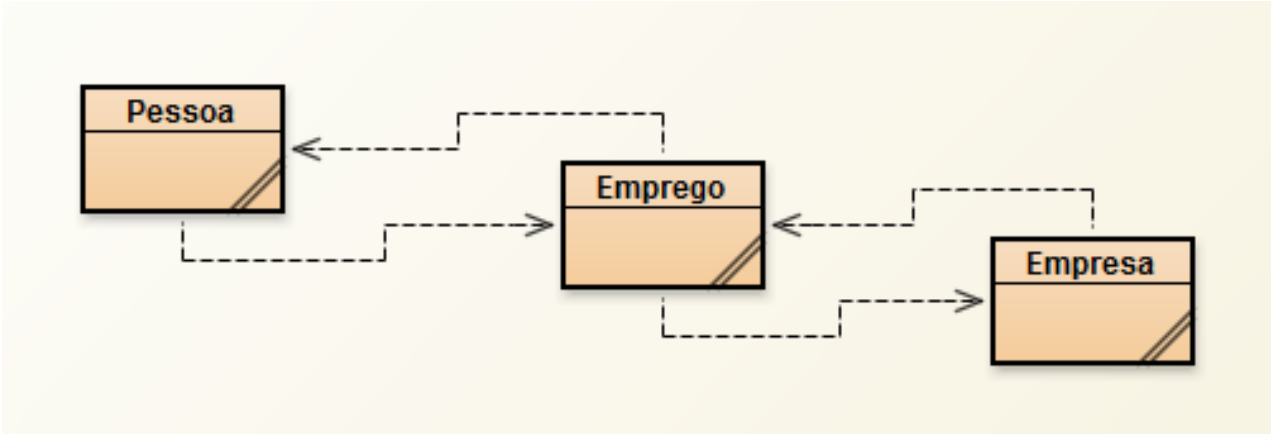
```
public class Pessoa {
   Conta conta;
   Endereço reside;
   ...
   void Pessoa(Endereço endereço) {
        conta = null;
        reside = endereço;
         ...
   }
}
```

```
public class Endereço {
   String rua;
   ...
}
```

Classe Associativa



Classe Associativa



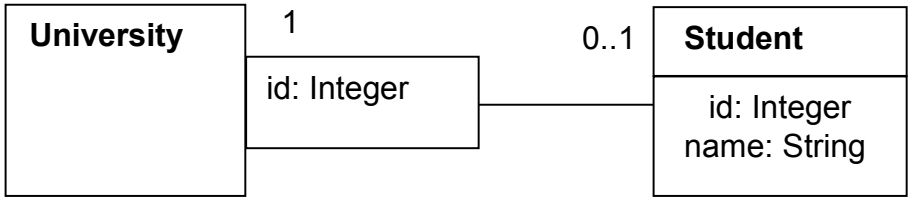
Classe Associativa

```java
import java.util.ArrayList;

public class Empresa
{
    private ArrayList<Emprego> empregos;

    public Empresa(){
        empregos = new ArrayList<Emprego>();
    }
}
```

```java
import java.util.ArrayList;

public class Pessoa
{
    private ArrayList<Emprego> empregos;

    public Pessoa(){
        empregos = new ArrayList<Emprego>();
    }
}
```

```java
import java.util.Date;

public class Emprego
{
    private Empresa empregador;
    private Pessoa funcionario;
    private double Salario;
    private Date dataInicio;

}
```

Associação Qualificada

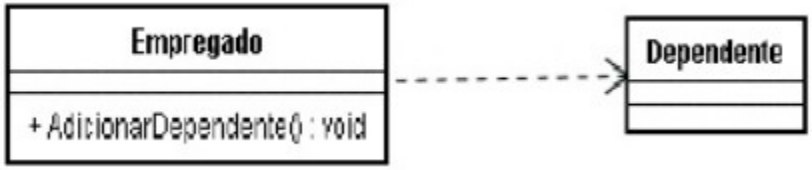



```java
import java.util.Hashtable;

public class University
{
    private Hashtable students;

    public void addStudent(Student s){
        students.put(s.getId(), s);
    }

    public void removeStudent(int id){
        students.remove(id);
    }

    public Student getStudent(int id){
        return (Student)students.get(id);
    }
}
```

```java
public class Student
{
    private int id;
    private String nome;

    public int getId(){
        return id;
    }
}
```

Implementação - Dependência

Dependência

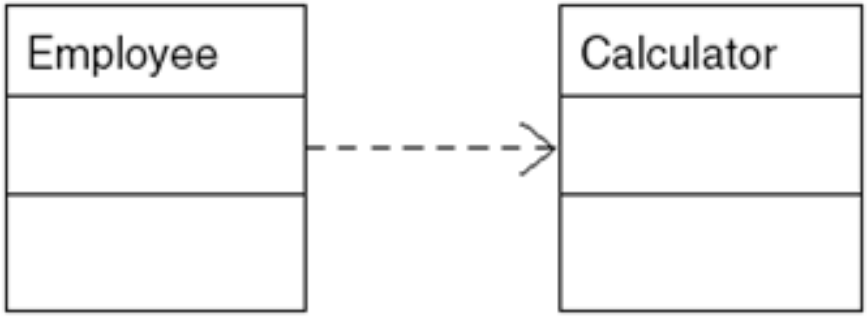



```
public class Empregado {

    public void adicionarDependente(Dependente dependente){
        //implementação do método
    }
}
```

Dependência por atributo

Dependência

```
public class Employee {
        public void calcSalary(Calculator c) {
        ...
        }
}
```



Referências

- Material da professora Bárbara de Melo Quintela (UFJF)
- BARNES, David J.; KÖLLING, Michael. Programação orientada a objetos com Java: uma introdução prática usando o BlueJ. 4 ed. São Paulo: Pearson Prentice Hall, c2009. ISBN: 9788576051879